Take-Home Activity – Personal Expense Tracker
Google Developer Groups on Campus - De La Salle University (Manila)
2025 - 2026

Overview

For this take-home activity, your task is to create a simple web application for personal expense tracking. The specification describes the minimum features required to be implemented for the project. Additional features are listed in the bonus section.

This activity demonstrates your understanding of basic web development with complete CRUD (Create, Read, Update, Delete) operations and proper database transaction management. The project must be completed individually.

This take-home activity is a requirement for all officers under the Software Development Committee of the Tech Department. For officers from other committees, completing this activity is a bonus accomplishment for you.

Since this event is incentivized, completion of this take-home activity may be required for the event to be considered for incentives. Please ensure timely submission of your output.

Project Specifications

The following describes the features of a personal expense tracking web application. The application should help users monitor their income and expenses, categorize transactions, and view spending patterns. The minimum features required to be implemented for this project are as follows:

General *(Optional but recommended for incentive output, otherwise required for GDGoC Software Development Officers)*

- Good user experience. Users can easily navigate without help, all information is easy to access
- Good visual design. Design suits the theme of the application, and is cohesive and consistent across the whole application
- Responsive layout that works well on different screen sizes
- Proper error handling with user-friendly error messages
- Input validation on both client and server side
- Protection against common security vulnerabilities (SQL injection, XSS, CSRF)
- Proper database transaction management ensuring data integrity

View Dashboard

- Upon visiting the web page, a user may see a simple dashboard displaying:
  - Current balance (total income minus total expenses)
  - A list of all transactions (showing the most recent first)

Add Transaction

- A user can add a new transaction with the following details:

© GDGOC 2026
Made by Leon Pavino

- Transaction type (Income or Expense)
- Amount (must be a positive number)
- Category (from a predefined list: Food, Transportation, Salary, Utilities, Entertainment, Others)
- Description (text field for additional details)
- Date of transaction (defaults to current date, but can be changed)
    - The transaction should be immediately reflected in the dashboard and balance calculations

View Transaction

- Users can view a complete list of all their transactions
- Transactions should display:
    - Date
    - Type (Income/Expense with visual indicator like color or icon)
    - Category
    - Description
    - Amount
        - Default sorting should be by date (most recent first)

Edit Transaction

- Users can edit any transaction
- All fields (type, amount, category, description, date) should be editable
- Changes should immediately update the dashboard and balance calculations

Add Transaction

- Users can delete any transaction
- A confirmation prompt should appear before permanently deleting a transaction
- Deleted transactions should be immediately removed from the database
- Balance calculations should update automatically after deletion

© GDGOC 2026
Made by Leon Pavino

Bonus Features (Optional)

The following features are not required but will earn additional learning if implemented properly:

Pagination

- Display transactions in pages (e.g., 20 transactions per page) instead of showing all at once
- Users can navigate between pages

Custom Categories

- Users can create new custom categories
- Users can edit or delete custom categories (cannot edit default system categories)
- If transactions exist for a category being deleted, prevent deletion or prompt to reassign

Search and Filter

- Users can search for transactions by:
  - Description text (partial match)
  - Category
  - Date range (from date to date)
  - Transaction type (Income/Expense)
    - Search results should show the total income and expenses for the filtered set

View Reports

- Users can view simple spending reports:
  - Expenses by category (showing total per category)
  - Income vs. Expenses comparison
    - Reports can be filtered by date ranges

Export Data

- Users can export their transaction data to CSV format
- Exported file should include all transaction fields (date, type, category, description, amount)

Advanced Features (not recommended; but if you want a challenge)

- Edit timestamps (show “Last edited on [date]” for modified transactions)
- Running balance column in transaction list
- Sorting options (sort by amount, category, or type)
- Dashboard date range filter (this week, this month, last 30 days, custom range)
- Payment method field (Cash, Credit Card, Debit Card, Bank Transfer)

© GDGOC 2026
Made by Leon Pavino

Important Notes

CRUD Operations: This project must implement complete CRUD (Create, Read, Update, and Delete) operations for all major entities:

- **Create**: Add Transaction
- **Read**: View Dashboard and View Transaction
- **Update**: Edit Transaction
- **Delete**: Delete Transaction

Database: Use a database to store transactions persistently. Data should not be lost when the application restarts.

No authentication required, this is a single-user application. No login/registration system is needed.

Since this is an individual activity, your code should be your own work & responsibility. You may reference documentation and tutorials, but direct copying of complete solutions is not permitted in the context of learning and development of this activity. Proper comments and understanding of your implementation should be evaluated by yourself, or others for a peer-to-peer review and learning.

Technical Requirements

1. Use appropriate web technologies (HTML, CSS, JavaScript)
    a. You can use other libraries/frameworks
2. Implement server-side logic with proper routing
3. Use a database system (e.g., MongoDB, MySQL, PostgreSQL) with proper schema design
4. Implement secure password hashing (e.g., bcrypt)
5. Use session management for user authentication
6. Follow MVC (Model-View-Controller) or similar architectural pattern
7. Include proper handling and validation
8. Write clean, well-commented code

© GDGOC 2026
Made by Leon Pavino

Submission Guidelines

If you want to publish this in your portfolio/GitHub repository, you can include:

1. Complete source code with proper folder structure
2. Database schema documentation
3. README file with:
    a. Setup instructions
    b. Database configuration steps
    c. List of implemented features
    d. Any additional features beyond minimum requirements
    e. Known issues or limitations
4. Sample credentials for testing (if applicable)

© GDGOC 2026
Made by Leon Pavino